

What is Zeek? Zeek is a passive, scriptable network analysis framework. It parses protocols in real time, generates structured logs, and lets you write custom scripts to detect threats.

Zeek vs Arkime (quick):

- Arkime = fast PCAP search and storage.
- Zeek = deep protocol analysis + custom scripting for alerts.

Companies often run both together.

Zeek System Workflow

1. Packets arrive → Event Engine creates events (http_request, dns_request, etc.).
2. Policy scripts react to those events.
3. Notices, logs, or actions are generated.

Key Directories:

- /usr/local/zeek/share/zeek/base/ — core frameworks.
- /usr/local/zeek/share/zeek/policy/ — built-in policies.
- /usr/local/zeek/share/zeek/site/ — your custom scripts (local.zeek).

Basic Commands:

```
Bash
zeek -r capture.pcap local.zeek # Replay PCAP with your script
zeek -i eth0 # Live capture
zeekctl deploy # Start cluster after editing node.cfg
zeekctl status # Check running status
```

Writing Zeek Scripts – Structure & Keys

Basic Script Template (with line-by-line comments):

```
zeek
@load base/protocols/http # Load the HTTP protocol framework so we can use its events
module MyModule; # Optional: organizes your script into a module (good practice)
export { # Export section makes things visible to other scripts
    redef enum Notice::Type += { Suspicious_HTTP }; # Add a new notice type so we can raise alerts
}
event http_request(c: connection, method: string, original_URI: string, ...)
# This event fires every time Zeek sees an HTTP request
{
    if ("password" in original_URI) # Check if the word "password" appears in the URI
    {
        NOTICE([$note = Suspicious_HTTP, # Raise a notice/alert with our custom type
            $msg = "Possible password in URI", # Human-readable message
            $conn = c]); # Attach the connection details
    }
}
```

Key Elements:

- @load — includes frameworks or other scripts.
- event — handler that runs when something happens (e.g., http_request).
- NOTICE — raises alerts (shows in notice.log).
- redef — modifies built-in constants or enums.

Data Types You Need:

- set[addr] — unique IPs.
- table[addr] of count — counters per IP.
- vector of string — lists.

Events & Handling

Most Useful Events:

```
zeek
event new_connection(c: connection) # Any new connection
event http_request(...) # HTTP request
event dns_request(...) # DNS query
event ssh_login_attempt(...) # SSH login try
event connection_state_remove(c: connection) # Connection ended
```

Example: Count failed SSH logins (with comments):

```
zeek
global ssh_fails: table[addr] of count &default=0; # Table that counts failures per IP address
event ssh_login_attempt(c: connection, success: bool)
# This event fires every time an SSH login attempt happens
{
    if (!success) # If the login failed
        ssh_fails[c$Id$orig_h] += 1; # Increase the failure count for this source IP
    if (ssh_fails[c$Id$orig_h] > 5) # If failures exceed threshold
        NOTICE([$note = SSH_Brute_Force, # Raise alert
            $msg = "Brute force detected",
            $conn = c]);
}
```

Notice Framework (Alerts)

How to raise a notice:

```
zeek
NOTICE([$note = My_Alert, # The type of notice
    $msg = "Something suspicious", # Human-readable message
    $conn = c]); # Attach connection details
```

Notices go to notice.log and can be emailed or sent to SIEM.

Logging Framework

Basic log creation (with comments):

```
zeek
redef enum Log::ID += { LOG }; # Add a new log stream ID
type Info: record { # Define what fields will be in the log
    ts: time &log; # Timestamp (logged)
    src: addr &log; # Source IP (logged)
    msg: string &log; # Message (logged)
};
event zeek_init() # Runs once when Zeek starts
{
    Log::create_stream(LOG, [$columns=Info, $path="mylog.log"]); # Create the log file "mylog.log"
}
```

Write to log:

```
zeek
Log::write(LOG, [$ts=network_time(), $src=c$Id$orig_h, $msg="Alert"]); # Write a record to the log
```

Monitoring DNS (Common Example)

```
zeek
event dns_request(c: connection, query: string)
# Fires every time Zeek sees a DNS query
{
    if (length(query) > 50 || /\.ru$/ in query) # High entropy or suspicious TLD
    {
        NOTICE([$note = Suspicious_DNS,
            $msg = "Possible C2 DNS",
            $conn = c]);
    }
}
```

Best Practices & Optimization

Script Writing Rules:

- Keep scripts small and focused.
- Use @load only for what you need.
- Always test with zeek -r small.pcap local.zeek.
- Add thresholds to avoid noise (e.g., >5 failed logins).
- Comment clearly.

Tuning Tips:

- Use &default=0 on tables.
- Avoid tight loops.
- Use timers for cleanup.
- Recompile only when needed (zeekctl restart).

Example Optimized Script (failed SSH) — with comments:

```
zeek
global ssh_fails: table[addr] of count &default=0; # Counter per IP, default 0
event ssh_login_attempt(c: connection, success: bool)
# Fires on every SSH login attempt
{
    if (!success) # If login failed
    {
        ssh_fails[c$Id$orig_h] += 1; # Increment failure count for this source IP
        if (ssh_fails[c$Id$orig_h] > 5) # If more than 5 failures
            NOTICE([$note = SSH_Brute_Force, # Raise alert
                $msg = "Brute force detected",
                $conn = c]);
    }
}
```

Zeek Base Protocols & Frameworks – Quick Reference

1. Zeek Base Protocols

These are the core protocol parsers. You @load them to get events for common traffic.

Most Important Ones You Need to Know:

Protocol	What it does	Common Use in Scripts
http	Parses HTTP requests/responses	Detect suspicious URLs, POSTs, User-Agents
dns	Parses DNS queries and responses	Detect C2, tunneling, high-entropy domains
ssh	Parses SSH connections and login attempts	Detect brute force
ssl / tls	Parses TLS handshakes and certificates	Detect unusual JA3, self-signed certs
smtp	Parses email traffic	Detect spam or data exfil via email
ftp	Parses FTP commands and file transfers	Detect data exfiltration
rdp	Parses Remote Desktop Protocol	Detect brute force or lateral movement

How to load them:

```
zeek
@load base/protocols/http
@load base/protocols/dns
@load base/protocols/ssh
```

2. Zeek Base Frameworks

These provide the tools and structure for writing useful scripts (noticing, logging, statistics, etc.).

Most Important Frameworks You Need to Know:

Framework	What it does	Common Use in Scripts
notice	Generates alerts/notices	Raise alerts (e.g., brute force, C2)
logging	Creates custom log files	Write your own structured logs
signatures	Signature-based detection	Load .sig files for simple pattern matching
sumstats	Tracks statistics and thresholds	Count events (e.g., failed logins) and alert
cluster	Manages distributed Zeek deployment	Run Zeek across multiple sensors
broker	Communication between Zeek nodes	Share data between workers and managers
input	Reads external files (CSV, JSON, etc.)	Load IOC lists or config files
intelligence	IOC matching (IPs, domains, etc.)	Match traffic against known bad indicators
analyzer	Core protocol analysis engine	Enable/disable analyzers

How to load them:

```
zeek
@load base/frameworks/notice
@load base/frameworks/logging
@load base/frameworks/sumstats
```

Most Common Pattern You Will Use:

```
zeek
@load base/protocols/http # Get HTTP events
@load base/frameworks/notice # Get the ability to raise alerts
event http_request(...) # React to HTTP traffic
{
    NOTICE([...]); # Raise an alert
}
```